

The Network Partition Boundary: Standalone Architectural Treatment of Partition Events in the Coordination Knowledge Substrate Pattern

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one boundary case — **network partition** — as a standalone architectural treatment, articulating how a CKS-coherent deployment behaves when network connectivity is disrupted such that substrate components are isolated from each other or substrate is isolated from composition partners, and how the resulting state divergence is reconciled upon reconnection.

Abstract

Network partition is a routine operational event in distributed deployments. When connectivity is disrupted between substrate components, or between substrate and composition partners, each side may continue operating against locally available substrate state, and the resulting state may diverge. The dominant treatment in distributed-systems practice is automatic consensus — Paxos-family protocols, Raft, vendor-managed failover with automatic merge, last-write-wins reconciliation — which converges divergent replicas without human involvement. CKS rejects this treatment because it instantiates contradiction collapse by automation: the architecture's commitment to conflict as first-class substrate object (§5) is violated when divergent state is automatically merged. This note formalizes the alternative treatment the source paper's commitments imply. During partition, each side operates per orchestration rules that specify partition-aware behavior, and the partition itself is recorded as bounded non-determinism in provenance metadata. Upon reconnection, divergent state is registered as first-class conflict with provenance from each side preserved, and reconciliation is handled by human-authored rules. The note states the boundary scenario, identifies the architectural commitments stressed, articulates the treatment, enumerates the anti-pattern treatments the architecture forbids, and provides the operational test.

1. Why the network partition boundary needs to be formalized as standalone

Network partition is the most operationally common event in distributed deployments that produces concurrent state divergence. When two substrate hosts cannot communicate, or when a substrate cannot reach a composition partner, neither side knows what the other is recording. If both sides continue accepting writes, the writes diverge; reconnection produces two histories that need to be brought together. The architectural question is what the substrate pattern says about this event, both during the partition and upon resolution, and the question is consequential because the dominant industry treatment is incompatible with the source paper's commitments.

The dominant treatment is automatic consensus. Distributed-systems practice has produced sophisticated algorithms — Paxos and its descendants, Raft, eventually-consistent replication with last-write-wins, vendor-managed failover that performs automatic merge upon healing — that converge divergent replicas without human involvement. The treatment is appropriate for deployments whose primary commitment is consistency-as-property; it is not appropriate for deployments whose primary commitment is governance-visibility-of-divergence. CKS belongs to the second category. The conflict-preservation commitment in §5 explicitly forbids automatic resolution of contradiction by any non-human-authored process; partition produces contradiction; therefore partition reconciliation cannot be automatic in the CKS sense.

Naming the boundary as standalone matters because partition has operational structure of its own — bidirectional divergence, reconnection events, partition-aware cell behavior — that the broader conflict commitment does not specify in detail, and because the standalone framing forecloses the misreading that CKS substrates can be operated under standard distributed-systems consistency protocols on the strength of those protocols' maturity. The misreading is wrong; the protocols solve a different problem.

2. The partition scenario and what makes it non-obvious

Network connectivity is disrupted such that substrate components on different sides of the partition cannot communicate, or such that substrate cannot reach composition partners. The disruption may be transient or persistent. During the partition, each side may continue operating using locally available substrate — reading, writing, executing cells under orchestration rules. Upon reconnection, the state on each side may have diverged: each side has accepted writes the other has not seen, and the two sides may have written content that contradicts.

What makes the scenario non-obvious is the bidirectional structure of the divergence. A composition partner failure is unidirectional: the partner is unavailable; the substrate continues to operate; the partner's state may exist on its own terms but is not visible to the substrate during the failure. A network partition is different. Both sides continue operating under the assumption that they are the authoritative copy, because each is locally operational and each is exchanging no signals with the other. The result is two histories with potentially conflicting writes, both authorized by the orchestration rules that governed each side at the time, and neither straightforwardly preferable to the other. The architecture must specify how this is handled without delegating the decision to an automated process.

A second non-obvious property is that the partition event itself is part of what must be recorded. If the substrate carries no record that a partition occurred, post-reconnection auditors looking at the merged

state cannot reconstruct why two contradictory writes coexisted, why one orchestration rule fired on one side without firing on the other, or why a coordination event recorded on one side has no echo on the other. Partition is a category of bounded non-determinism the substrate must surface, not silence.

3. Architectural commitments stressed by the partition boundary

The boundary stresses five commitments simultaneously, and the treatment must respect all five.

Conflict as first-class object. Partition produces conflicts: divergent writes on the two sides, conflicting decisions made under the same orchestration rule applied to two different local states, contradicting facts recorded on each side. These are precisely the substrate content that §5 requires to be treated as first-class, addressable, persisted-by-default objects with their own provenance, produced bidirectionally and at potentially large volume upon reconnection.

The determinism contract within each partition. Within each side of the partition, the substrate's determinism guarantees still apply: reads of identical local state yield identical answers; writes are addressable; conflicts within the partition are preserved; the local substrate is the source of truth for the local side. The contract does not require global determinism across the partition (the partition itself defeats that); it does require local determinism on each side. Cell-behavior determinism follows the same scope.

Path retraceability across partition events. The partition event itself, the writes accepted on each side, and the reconciliation event upon reconnection must all be reconstructable from substrate content alone. The provenance fields — writer, timestamp, orchestration rule, antecedent content, rationale, and the architectural-context field — must distinguish which side's substrate authored each partition-era write, and the provenance must persist through reconciliation so that audit can read partition history out of the merged substrate.

Rule authoring for partition-aware behavior. Orchestration rules must specify what cells do during a partition. The default of "behave as if the partition does not exist" is not architecturally available, because such a default would have cells make decisions assuming connectivity that does not hold, producing writes inconsistent with the rule's actual conditions of correctness. Partition behavior is a rule-authoring concern, exercised at the source paper's Moment 1, not a runtime concern that fixes itself.

Partition events as bounded non-determinism. A partition is a category of non-determinism that affects when and how substrate operations complete; it is bounded in the sense that the substrate's partition-aware rules constrain what may happen during the event. The bounded non-determinism is recorded in provenance, not silenced.

The treatment must satisfy all five.

4. The architectural treatment

The treatment has three phases.

During the partition: local operation under partition-aware rules. Each side continues operating using locally available substrate state. The orchestration rules must specify, for each class of operation, what cells do when cross-partition coordination is required but unavailable. The rules may direct the cell to proceed

locally (recording the partition status in provenance), to queue the operation for cross-partition coordination upon reconnection, to fail the operation with a structured error that itself becomes substrate content, or to follow other patterns the rule author has anticipated. The architectural commitment is not that any one of these patterns is correct in general; it is that the pattern is specified by human-authored rules in advance, not selected by the runtime, the LLM, or the vendor. Partition events are recorded in the substrate as bounded non-determinism, with full provenance fields including which side originated the write.

At reconnection: divergent state registered as first-class conflict. When connectivity returns and the two sides exchange substrate state, divergent content is not merged. It is registered as conflict — substrate-level first-class conflict objects, each with its own identity, provenance from both sides, and addressability. Provenance distinguishes which side's substrate authored which writes, so that audit reading the substrate after reconnection can reconstruct the partition history without reference to logs external to the substrate. No automatic merge occurs. No last-write-wins selection occurs. No LLM-mediated arbitration occurs. The conflicts are preserved as substrate state until reconciliation occurs.

During reconciliation: human-authored rules under rule-conflict-resolution semantics. Humans author reconciliation rules and apply them to the registered conflict objects, either directly (Moment 2 — direct override) or indirectly (through orchestration rules that specify reconciliation under specified conditions). Both sides' state is preserved through reconciliation: the original divergent writes from each side, with their original provenance, remain in substrate as the antecedents the reconciliation operates on; reconciliation produces new substrate state with its own provenance, which references the antecedents rather than overwriting them. A reader of the post-reconciliation substrate can still see what each side wrote, who reconciled it, under what rule, and with what rationale. The reconciliation is itself a substrate write, with full provenance, recorded as a first-class governance event.

The treatment trades automatic consistency for governance visibility. The trade is deliberate: a deployment that hides partition divergence behind automatic merge satisfies a different commitment (consistency-as-property) than the one CKS makes (governance-of-divergence-as-substrate-property), and the two cannot be satisfied simultaneously by the same architectural move.

5. Anti-pattern treatments that would violate the architecture

Eight anti-patterns recur across systems that approximate CKS without satisfying the partition boundary.

Auto-merge-on-reconnection. The runtime, the substrate platform, or a middleware layer automatically merges divergent state upon reconnection, without producing first-class conflict objects and without involving human-authored rules. This is the canonical instantiation of contradiction collapse by automation at the partition boundary.

Last-write-wins reconciliation. The system selects between contradictory writes by clock comparison or some other automatic criterion. The selection is automatic and not part of human-authored orchestration rules; both the conflict-preservation and rule-authoring commitments are violated.

LLM-mediated conflict resolution. Upon reconnection, an LLM is invoked to read the divergent state and produce a reconciled state. This assigns the LLM the substrate-mediator-plus-judge role that §5.3 specifically excludes; the AI-as-substrate-mediator commitment is violated.

Vendor-managed consensus algorithms. A consensus protocol — Paxos, Raft, or any descendant — operating outside human-authored orchestration rules determines the post-partition substrate state. The protocol's election, leader, log-replication, and conflict-resolution semantics are vendor- or protocol-defined, not substrate-governed. The protocols are mature and may be useful adjacent infrastructure; what they cannot do is determine substrate reconciliation under the CKS commitments.

Partition-state-discarded-on-reconnect. One side's writes during the partition are discarded — typically the side that lost an election, the side judged to be the minority partition, or the side the vendor's failover criteria deselect. The discarded writes had provenance, were authorized by the orchestration rules that governed that side, and represented governance events on their own terms. Discarding them violates path retraceability and conflict-as-first-class.

Silent divergence. The partition event is not recorded in substrate; cells continue operating without their writes being marked with partition provenance; upon reconnection, the system has no record that a partition occurred. Subsequent audit cannot distinguish partition-era writes from normal writes, cannot reconstruct which side authored what, and cannot detect divergence that survived reconnection.

Auto-shedding during partition. Cells silently drop operations during the partition rather than executing them under partition-aware rules. The dropped operations are not recorded; the rule-authoring and retraceability commitments are violated together.

Vendor-managed failover during partition. The vendor's failover machinery — promotion of a standby replica to primary, redirection of traffic to a different region, automatic activation of a disaster-recovery substrate — performs state operations outside the substrate's governance. The new state may be internally consistent, but its provenance derives from the vendor's failover policy, not from human-authored orchestration rules over substrate content.

The eight anti-patterns share a common shape: each substitutes an automated process for a human-authored rule at the architectural slot where the source paper requires the human-authored rule.

6. Operational implications

Rules must specify partition behavior. A deployment whose orchestration rules do not address partition is not partition-coherent, because cells operating during a partition are operating without rules that contemplate the partition condition, and any partition-era writes they produce are governed by rules whose conditions of validity do not hold. Authoring partition-aware rules is part of what it means to author orchestration rules at all in a deployment subject to network partition.

Reconciliation is a human-authored governance event. Reconciliation upon reconnection is exercised at the source paper's Moment 1 (rule authoring) or Moment 2 (direct override). Both moments are human governance events; neither is a runtime decision. The reconciliation's cost is paid at these moments and not at substrate-size scale, preserving the linear-cost commitment.

Both sides' history is preserved. A reader of the post-reconciliation substrate can reconstruct the partition history by reading substrate content alone. This is what makes the substrate audit-bearing across partition events, not merely consistent across them.

7. Limits of the architectural treatment

The boundary case applies specifically to network partition affecting substrate components or substrate-to-composition-partner connectivity. It does not apply to vendor-level outages where partition is not the relevant abstraction, to composition partner failures that do not involve bidirectional substrate divergence (covered by the composition-partner-failure boundary), to LLM consultation timeouts or failures (covered by the LLM-consultation boundary), or to single-node failures within a substrate that do not produce partition. The treatment is also silent on the choice of network protocols, replication topologies, or specific platforms that produce or detect partition events; those are deployment decisions, not architectural ones.

8. The operational test

A CKS deployment satisfies the partition boundary's architectural treatment if and only if all of the following are true at all times during the substrate's existence:

1. **Partition-aware rules.** Orchestration rules specify, for each class of operation, what cells do when cross-partition coordination is required but unavailable.
2. **Partition events recorded.** Partition events are recorded as bounded non-determinism in provenance, with the side of origin identified for each partition-era write.
3. **No automatic merge.** Divergent state upon reconnection is registered as first-class conflict; no automated process — vendor consensus, LLM judge, last-write-wins selector, or middleware merge — produces the post-reconnection state.
4. **Both sides preserved.** The original divergent writes on each side, with their provenance, remain in substrate as the antecedents reconciliation operates on.
5. **Human-authored reconciliation.** Reconciliation is performed by humans directly (Moment 2) or by orchestration rules authored under human governance (Moment 1); the reconciliation event is recorded as a first-class substrate write with full provenance.

A deployment that fails any of (1)–(5) may still recover from network partitions in some operational sense, but is not partition-coherent in the CKS sense.

9. Why naming the boundary as standalone matters

The network partition boundary is one note in a Phase A6 program that formalizes boundary cases the architecture's foundational commitments must hold across. The cumulative effect of the program is that CKS as a design pattern carries an explicit specification of behavior under operational stress, not only in the canonical case the foundational notes treat. A reader who encounters CKS in a deployment where network partition is a real operational concern needs to be able to point to a single specification of how the architecture treats partition, and the specification needs to foreclose the misreading that automatic consensus protocols satisfy CKS by virtue of being mature distributed-systems primitives. Subsequent boundary-case notes formalize further cases — vendor data-handling-policy change, cell timeout and long-running operation behavior, and others — each on the same pattern.

A reader familiar with distributed-systems literature will notice that this treatment is in some operational respects less convenient than the consensus-protocol alternative. That is the trade the architecture makes. Consensus protocols converge replicas to a consistent state without operator involvement; CKS preserves divergence as substrate content the operator can read. The two patterns answer different questions about what a substrate is for. CKS commits to the substrate-as-source-of-truth-for-what-was-decided answer (§11.3); a partition during which decisions were made on each side is part of what the substrate must carry truthfully, not part of what it should hide for the operator's convenience.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Network Partition Boundary: Standalone Architectural Treatment of Partition Events in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.